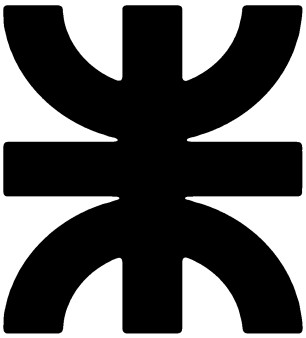
Universidad Tecnológica Nacional

Facultad Regional Córdoba

Ingeniería en Sistemas de Información



TEMA: TDD

Docentes:

- Ing. Cecilia Massano
- Ing. Meles, Judith
- Ezequiel Izaguirre
- Marcos Pomenich

Curso: 4K2
Grupo Nro: 3
Integrantes:

- Bisio Juan Martin - 401571
- Caffaratti, Máximo David - 401609
- Crovato Agustin - 403878
- Lavini Alejandro Raul - 402745
- Fava, Augusto - 95058
- Giorgi, Lorenzo - 403852
- Maldonado Alberich Gonzalo - 404483
- Mansur, Nicolas - 402774
- Martínez, Joaquín - 96110
- Matos, Kevin Lionel - 400734
- Pereyra, Santiago - 404539
- Portalupi Matías - 403916
- Ques, Fermín - 402066
- Quintana Tomás - 402992
- Tarifa, Angela - 94599

Justificación de decisiones de diseño – Grupo 3 - 4K2

User Story seleccionada

Para este trabajo práctico se seleccionó la User Story **"Comprar entradas"** del sistema **EcoHarmony Park**, que le corresponde a los grupos impares. La funcionalidad permite que un visitante registrado compre entradas indicando la fecha de visita, la cantidad, la edad y el tipo de pase de cada visitante (común o VIP) y la forma de pago, que puede ser efectivo en boletería o tarjeta con Mercado Pago.

Aplicación de TDD

La funcionalidad se desarrolló aplicando **TDD**, siguiendo el ciclo **Red-Green-Refactor**.

Primero se escribieron las pruebas automatizadas con **pytest** y el cliente de pruebas de Django, que simulan lo que hace un visitante en la web: registrarse, iniciar sesión y completar el formulario de compra. (RED)

Después se escribió el código mínimo para que esas pruebas pasaran: rutas, vistas, plantillas, el modelo de compra y las reglas de validación. (GREEN)

Por último se ordenó el código sin cambiar su comportamiento, sacando la validación a funciones más chicas y verificando que las pruebas siguieran en verde. (REFACTOR)

Se hicieron dos ciclos. En el primero se escribieron nueve pruebas de la compra por web, que fallaron con error 404 porque las rutas todavía no existían; luego se armó la aplicación y las nueve pasaron. En el segundo, después de aclarar las reglas con el PO, se agregaron pruebas para los días de cierre, el límite de un mes de anticipación, el horario de corte de las 17:30, los descuentos por edad y el tope de entradas por fecha, y se ajustó el código hasta que volvieron a pasar.

En total son 30 pruebas, todas aprobadas. Dos de ellas (las del horario de corte del mismo día) se omiten solas si se ejecutan un día en que el parque está cerrado. Las pruebas de usuario del enunciado quedaron cubiertas así:

- Compra válida con tarjeta o efectivo: `test_compra_con_tarjeta_muestra_checkout_simulado` y `test_compra_en_efectivo_muestra_confirmacion_de_mail_simulado`.
- Sin forma de pago: `test_compra_sin_medio_de_pago_muestra_error_y_no_crea_compra`.
- Parque cerrado: `test_compra_rechaza_los_lunes` y `test_compra_rechaza_fechas_festivas`.
- Más de 10 entradas: `test_compra_rechaza_mas_de_diez_entradas` y `test_compra_rechaza_mas_de_diez_entradas_acumuladas_por_fecha`.

Decisiones de diseño tomadas

Se decidió separar las reglas del negocio de las vistas. Todo lo que valida la compra está en `services.py` y las vistas solo reciben el pedido, llaman al servicio y devuelven la respuesta. La validación se partió en funciones chicas, cada una con una sola tarea: `_validar_fecha_visita`, `_validar_cantidad_entradas`, `_validar_visitante` y `_precio_con_descuento`, que se usan desde `validar_compra`. Así el código es más fácil de leer y de probar, y si una regla cambia se toca un solo lugar.

El formulario avisa los errores en el navegador (calendario con fecha máxima, campos numéricos, total estimado), pero el servidor vuelve a validar todo y recalcula el total, para no depender de lo que mande el cliente. Los errores se juntan en una excepción propia, `ErrorValidacionCompra`, así se muestran todos juntos, y el resultado correcto se devuelve en `CompraValidada`.

Las pruebas usan las rutas reales de la aplicación y una base de datos en memoria, de modo que se prueba lo que ve el visitante y no solo funciones sueltas. Además `validar_compra` recibe la fecha y la hora como parámetros opcionales, lo que permite probar el horario de las 17:30 sin depender del reloj.

Las reglas que no estaban en el enunciado se consultaron con el PO:

- El parque cierra los lunes, el 25 de diciembre y el 1 de enero.
- Se compra desde hoy hasta un mes de anticipación; para el mismo día, hasta las 17:30.
- Máximo de 10 entradas por usuario y fecha, sumando todas sus compras. Se controla dentro de una transacción para que dos compras al mismo tiempo no pasen el límite.
- Pase común $10.000 y VIP $20.000. Hasta 3 años no pagan; de 4 a 15 años y desde los 60 tienen 50% de descuento.

Los precios no están escritos en las vistas, se leen de la configuración (`PRECIO_ENTRADA_REGULAR` y `PRECIO_ENTRADA_VIP`).

El pago con tarjeta y el mail de confirmación se simularon: el primero con una pantalla local que imita a Mercado Pago y el segundo con un mensaje “Mail enviado (simulado)”. Esto permite probar todo el recorrido sin servicios externos ni cobros reales. El envío real por SMTP quedó preparado en `enviar_confirmacion_compra` y se puede ejecutar a mano con el comando `reenviar_confirmacion`, con las credenciales en variables de entorno y no en el repositorio.

Como base de datos se usó SQLite a través del ORM de Django, con el modelo `Compra` (usuario, fecha, cantidad, visitantes, medio de pago, total y estado). Para probar entre todos se creó el comando `crear_usuario_compartido`, que genera la cuenta de prueba con una contraseña aleatoria.

Convenciones de código utilizadas

Se siguió la guía elegida en el TP2, **PEP 8**, la guía de estilo oficial de Python.

Nombres de variables, funciones y métodos

Se usó **snake_case** con nombres descriptivos. Por ejemplo:

- `validar_compra`
- `fecha_maxima_compra`
- `medio_pago`
- `entradas_existentes`

Las funciones de uso interno empiezan con guion bajo (por ejemplo `_validar_visitante`) para indicar que no son parte de la interfaz del módulo.

Clases y constantes

Las clases usan **PascalCase** (`Compra`, `CompraValidada`, `ErrorValidacionCompra`) y las constantes y valores de configuración **MAYÚSCULAS_CON_GUIONES** (`PRECIO_ENTRADA_VIP`, `EMAIL_COMPARTIDO`), para reconocer rápido que no deben modificarse.

Sangría y bloques de código

Python marca los bloques con la indentación, así que no se usan llaves. Se usaron 4 espacios por nivel, sin tabulaciones.

```
def _precio_con_descuento(precio, edad):
    if edad <= 3:
        return Decimal("0")
```

Longitud de línea, imports y espacios

El máximo es de 79 caracteres por línea, como pide PEP 8, y en Visual Studio Code se dejó una regla visual para verlo. Los imports están al principio de cada archivo, agrupados en librería estándar, Django y módulos propios. Se dejaron espacios alrededor de los operadores y después de las comas, y líneas en blanco entre funciones y clases.

Organización del código

El código se organizó separando responsabilidades:

- `services.py`: reglas de negocio, cálculo del total y envío del mail.
- `views.py`: registro, inicio y cierre de sesión, compra de entradas y pago simulado.
- `models.py`: modelo `Compra` y sus estados.
- `urls.py`: rutas de la aplicación.
- `templates/`: plantillas HTML de la aplicación.
- `management/commands/`: comandos de apoyo (usuario compartido y reenvío del mail).
- `tests/test_comprar_entradas.py`: pruebas automatizadas con pytest.
- `docs/`: concepto de la US, lista de pruebas, bitácora de ciclos y decisiones de diseño.

Esta separación hace que cada archivo tenga una función clara y que las reglas puedan probarse por separado.

Conclusión

Las decisiones tomadas permitieron construir una funcionalidad simple, clara y testeable.

El uso de TDD ayudó a verificar cada comportamiento antes de avanzar con la implementación y a refactorizar con la seguridad de que las reglas seguían funcionando.

Además, las convenciones de PEP 8 y la separación de responsabilidades mejoran la legibilidad y facilitan futuras modificaciones, como reemplazar las simulaciones por Mercado Pago y el envío de mail reales.